“Who do you think the enemy is?”

Attack on Titan — The Battle for Wall Rose

Authors: **Lakshya Tambi, Arnav, Rohan**

Duration	90 Minutes
Files	TitanShifter.java ScoutSquad.java WallDistrict.java BattleOfWallRose.java

Scenario

Wall Rose has been breached. Titans pour through the gaps and the Scout Regiment scrambles to mount a defence. Intelligence units decode radio signals in binary and hexadecimal. Combat squads assess readiness, engage hostile Titan Shifters, and — when outmatched — call upon the Coordinate ability of a friendly Shifter to command the pure titans. Your mission: implement the four Java classes below so the complete battle simulation compiles and runs correctly.

Class 1 — TitanShifter

Models a Titan Shifter tracked by military intelligence. Wrapper types are used deliberately throughout — understand how they behave during autoboxing, unboxing, and re-boxing.

Fields

String name | **Integer** powerLevel | **Double** heightMeters | **Boolean** isHostile

Constructors (Overloaded)

- TitanShifter(String name, String powerStr, String heightStr, String hostileStr) — Parse each string into its corresponding wrapper type using the appropriate valueOf() method.
- TitanShifter(String name, Integer powerLevel, Double heightMeters, Boolean isHostile) — Directly assign the typed parameters to the corresponding fields.

Getters & Setters

Implement standard getters and setters for all four fields: getName/setName, getPowerLevel/setPowerLevel, getHeightMeters/setHeightMeters, getIsHostile/setIsHostile.

Non-Static Inner Class — TitanForm

TitanForm is a non-static inner class. Because it is non-static, each instance is tied to an enclosing TitanShifter object and can directly access the outer class’s instance fields.

- public static final int MAX_TRANSFORM_LIMIT = 3; — Maximum transformations allowed before stamina depletion.
- TitanForm(String transformCountStr) — Parses transformation count from its string representation.
- TitanForm(int transformCount) — Overloaded constructor.
- getTransformCount() and setTransformCount(Integer count).
- Integer getPowerSurge() — Computes combat power surge as powerLevel × transformCount × 2 using the enclosing shifter’s powerLevel.

Static Nested Class — CoordinatePower

CoordinatePower is a static nested class. It does NOT have implicit access to an enclosing instance — therefore, it can only access outer-class **instance** fields via an explicit object reference (e.g., shifter.getIsHostile()). Note that static members of the enclosing class can be accessed directly. Tracks activeTitansControlled as a static Integer.

- static Integer getActiveTitansControlled() and static void setActiveTitansControlled(Integer count).
- static Boolean triggerCoordinate(TitanShifter shifter, String commandSignal) — Decodes binary command (radix 2). If the shifter is friendly (not hostile) and the decoded signal is greater than 0, sets activeTitansControlled = decodedSignal × 10 and returns true. Otherwise returns false.

Method — assessCombatReadiness (Provided)

static int[] assessCombatReadiness(Integer power) — *Provided in full for your conceptual understanding of the four-step autoboxing, unboxing, re-boxing, and non-reboxing sequence. Read it carefully before the evaluation.*

Class 2 — ScoutSquad

Models an elite combat squad in the Survey Corps. Uses overloaded constructors where one constructor calls another in the same class using this(...) (constructor delegation), varargs, and Boolean wrapper types.

Fields

String squadLeader | **String[]** members | **Integer** squadPower | **Boolean** deployed

Constructors (Overloaded & Varargs)

- ScoutSquad(String leader) — Single-leader squad. Calls the second constructor using this(...) by passing leader and an empty String array (calling one constructor from another in the same class using this(...) is known as constructor delegation).
- ScoutSquad(String leader, String... members) — Varargs constructor. Computes squadPower = members.length × 50 + 100 as an Integer. Sets deployed to the Boolean representation of false.

Getters & Setters

Implement getters and setters for all four fields: getSquadLeader/setSquadLeader, getMembers/setMembers, getSquadPower/setSquadPower, isDeployed/setDeployed.

Methods

- void boostSquadPower(Integer bonus) — Boosts squad power by the given bonus amount.
- boolean canEngage(TitanShifter titan) — Returns true only if the titan is hostile and squadPower ≥ titan.getPowerLevel().
- String[] getFullRoster() — Returns an array with the leader at index 0 followed by all members.
- static String reverseDistressSignal(String rawSignal) — Decodes inverted distress transmissions sent during Titan ambush. Reverses the order of words in the string, separated by single spaces, with no leading or trailing spaces.

  Example: "advance towards the forest" → "forest the towards advance"

  Example: " breach at wall rose " → "rose wall at breach"

  Hint: You may find String's trim() (e.g. " hello ".trim() → "hello") and split("\\s+") (e.g. "hello world".split("\\s+") → ["hello", "world"]) methods useful.

  Note: Evaluated independently — not invoked inside executeDefenseOperation.

Class 3 — WallDistrict

Represents a fortified district along Wall Rose (e.g. Trost, Krolva, Ehrmich). Handles district kill matrices, signal decoding, and titan identification.

Fields

String districtName | **Integer** wallIntegrity | **Integer[][]** titanKillMatrix | **TitanShifter** spottedTitan

Sector & Squad Matrix: Rows represent defence sectors along the district wall (Sector 0, Sector 1, …), columns represent assigned defence squads (Squad 0, Squad 1, …). For example, matrix[0][1] = 5 means Sector 0 had 5 confirmed kills credited to Squad 1. A null entry means that squad was not deployed there.

Constructors (Overloaded)

- WallDistrict(String name) — Default 100% wall integrity and empty kill matrix.
- WallDistrict(String name, String integrityStr, Integer[][] killMatrix) — Parses integrityStr to Integer and assigns the kill matrix. spottedTitan starts as null.

Getters & Setters

Implement getters and setters for all four fields: getDistrictName/setDistrictName, getWallIntegrity/setWallIntegrity, getTitanKillMatrix/setTitanKillMatrix, getSpottedTitan/setSpottedTitan.

Methods

- Integer calculateTotalConfirmedKills() — Traverses titanKillMatrix with nested loops, skipping null entries, and returns the total kill count as Integer.
- boolean verifyTitanIdentification(String reportId1, String reportId2) — Parses both report IDs to Integer and verifies whether they identify the same titan. *Hint: In Java, comparing objects with == checks memory references — think carefully about how object value equality actually works.*
- int[] decodeSignal(String binarySignal, String hexSignal) — Decodes binary signal (base 2) and hex signal (base 16). Returns [urgencyLevel, titanCount].

Class 4 — BattleOfWallRose (Master Coordinator)

The master tactical command system. Maintains arrays of registered WallDistrict, ScoutSquad, and TitanShifter objects (max 5, 10, and 10 respectively) and a running totalHumanityDefenseScore (Integer). The final method executeDefenseOperation ties together every class you have written.

Fields

WallDistrict[] districts | **ScoutSquad[]** activeSquads | **TitanShifter[]** encounteredTitans | **Integer** totalHumanityDefenseScore

Constructor & Registration Methods

- BattleOfWallRose() — Initialise all three arrays to their max sizes; set all counters and totalHumanityDefenseScore to 0.
- boolean addDistrict(WallDistrict district), boolean registerSquad(ScoutSquad squad), boolean registerTitan(TitanShifter titan) — Add the given object to its respective array if capacity is not exceeded; return true on success.

Getters

Implement getTotalHumanityDefenseScore(), getDistrictCount(), getSquadCount(), getTitanCount().

Overloaded Methods (Varargs)

- static int deploySquadUnits(int squadSize) — Exact-match overload; returns squadSize.
- static int deploySquadUnits(int firstSquad, int... otherSquads) — Varargs overload; returns the sum of all squad sizes.

Helper Method

- TitanShifter findFriendlyShifter() — Searches encounteredTitans for a shifter whose isHostile is false. Returns the first match, or null if none found.

Master Execution Method — executeDefenseOperation

boolean executeDefenseOperation(String commandSignal, String hexIntel)

Orchestrates the entire operation across all registered districts:

1. **Signal Intelligence:** For each district, call district.decodeSignal(commandSignal, hexIntel) and add the decoded urgency level and titan count to totalHumanityDefenseScore.

2. **Kill Tallying:** Call district.calculateTotalConfirmedKills() and add the result to the score.

3a. **Hostile Titan Spotted:** Search activeSquads for a non-deployed squad that canEngage the titan. If found, mark it deployed and add its squadPower to the score. If no squad can engage alone, find a friendly shifter via findFriendlyShifter(), create a TitanForm (count "2"), retrieve its getPowerSurge() to boost the first squad's power, mark it deployed, add its updated power to the score, and also run TitanShifter.assessCombatReadiness(friendly.getPowerLevel()) and add result[0] to the score.

3b. **Friendly Titan Spotted:** Call TitanShifter.CoordinatePower.triggerCoordinate(spottedTitan, commandSignal). If it returns true, add CoordinatePower.getActiveTitansControlled() to the score.

4. **Outcome:** Return true if totalHumanityDefenseScore ≥ 100 (Wall Rose secured); otherwise return false.

All the best!